

Rapport de mission technique 2eme année Amélioration du processus R&D



Yassine Hamrouni

Tuteur entreprise : Maxime Phan Thanh

Tuteur école : Johan Thapper

Sommaire

Glossaire

Introduction

I. Présentation générale de l'entreprise

II. Évolution socio-économique

1. Croissance internationale et diversification sectorielle
2. Impact sur l'emploi et les compétences
3. Contributions à la compétitivité industrielle
4. Responsabilité sociétale et développement durable

III – Présentation d'équipe

1. Service d'accueil

- 1.1. Missions principales
- 1.2 Objectifs du processus
- 1.3 Importance et accessibilité

2. Organigramme de l'équipe

- 2.1 Structure générale
- 2.2 Mon rôle spécifique

IV. Mes missions

1. Contexte et Enjeux

Contexte de la mission

Enjeux pour l'entreprise

3. Moyens et Ressources

Backend : Node.js + Express

Frontend : React

Base de données : PostgreSQL

4.1 Mise en place du middleware d'authentification

4.3 Sécurité des données

4.4 Développement du Frontend et gestion des événements

Authentification côté frontend

4.5 Gestion des événements et des données

L'application met en place différents useEffect pour :

1. Charger les logs d'actions (events) depuis l'API.
2. Charger dynamiquement les données (workstations, GPU, processeurs, salles, etc.) en fonction de l'URL visitée.
3. Mettre à jour les filtres, recherches et tris côté frontend sans recharger la page.

Exemple de chargement dynamique des entités :

Intégration avec l'API et gestion des logs

Toutes les actions utilisateur (consultation, ajout, modification, suppression) sont envoyées vers l'API.

L'API, de son côté, enregistre un log horodaté en base, associé au trigramme de l'utilisateur.

Exemple : lors de la consultation des logs, le frontend envoie une requête protégée

V. Bilan professionnel

Facilités observées

Difficultés rencontrées et moyens mis en œuvre

Compétences mobilisées et progression

Complémentarité école / entreprise

Perspectives

VI. Conclusion

Glossaire

API (Application Programming Interface) : Interface qui permet à différentes applications de communiquer entre elles via des services ou des fonctions.

Backend : Partie d'une application qui gère la logique métier, la base de données et les échanges avec le serveur.

Base de données relationnelle : Système de stockage des données sous forme de tables reliées entre elles par des relations (ex. PostgreSQL).

CAS (Central Authentication Service) : Protocole d'authentification unique (Single Sign-On) permettant de se connecter une seule fois pour accéder à plusieurs applications sécurisées.

Certification logicielle : Processus de validation garantissant que les logiciels fonctionnent correctement sur des configurations matérielles spécifiques.

CRUD (Create, Read, Update, Delete) : Ensemble des quatre opérations de base pour manipuler les données dans une base.

Excel : Logiciel tableur de Microsoft, utilisé pour manipuler et stocker des données sous forme de fichiers.

Express : Framework minimaliste pour Node.js, utilisé pour créer des serveurs web et des API REST.

Frontend : Partie visible d'une application web, correspondant à l'interface utilisateur.

GitLab : Plateforme de gestion de version et de CI/CD permettant de stocker du code, collaborer et automatiser le déploiement.

IDE (Integrated Development Environment) : Environnement de développement intégré regroupant des outils pour coder (éditeur, débogueur, compilateur).

Journalisation (Logs) : Enregistrement automatique des actions effectuées sur une application (qui, quoi, quand).

Jumeau numérique : Représentation virtuelle d'un produit ou processus permettant de le simuler et de l'optimiser.

KAO : Terme interne pour désigner une ou plusieurs erreurs remontées lors des tests logiciels.

Maintenance prédictive : Méthode qui anticipe les défaillances grâce aux données recueillies, afin d'optimiser les interventions.

Middleware : Composant logiciel intermédiaire qui gère les communications ou les traitements avant qu'une requête n'atteigne sa destination (ex. authentification).

Ngrok : Outil permettant d'exposer un serveur local sur Internet via une URL temporaire sécurisée en HTTPS.

Node.js : Environnement d'exécution JavaScript côté serveur.

ODT (tests logiciels) : Tests effectués par l'équipe software pour vérifier le bon fonctionnement des logiciels sur différentes configurations matérielles.

ORM (Object-Relational Mapping) : Technique qui permet de manipuler une base de données avec un langage de programmation orienté objet (ex. Sequelize).

PLM (Product Lifecycle Management) : Gestion du cycle de vie d'un produit, de sa conception à sa maintenance.

PostgreSQL : Système de gestion de base de données relationnelle open-source, robuste et performant.

React : Bibliothèque JavaScript utilisée pour construire des interfaces utilisateur dynamiques.

R&D (Recherche et Développement) : Activité de conception et d'innovation dans les entreprises technologiques.

Sequelize : ORM pour Node.js, facilitant l'interaction avec les bases de données relationnelles comme PostgreSQL.

SSO (Single Sign-On) : Système d'authentification unique permettant d'accéder à plusieurs services après une seule connexion.

Token : Jeton d'authentification utilisé pour sécuriser les communications entre un client et un serveur.

Workstation : Station de travail informatique dédiée à des tâches exigeantes (ex. conception 3D, simulation).

XML : Format de données structuré, souvent utilisé pour l'échange d'informations entre systèmes.

xml2js : Bibliothèque JavaScript permettant de convertir du XML en objets JavaScript.

Introduction

Au cours de ma troisième année d'apprentissage chez Dassault Systèmes, à Vélizy-Villacoublay, j'ai poursuivi la mission entamée l'année précédente dans le cadre de mon poste d'apprenti ingénieur informatique. Après avoir conçu une base de données et initié le développement d'une API sécurisée, j'ai pu, durant cette nouvelle période, finaliser l'API et proposer une première version fonctionnelle du site permettant son exploitation.

Cette année a été marquée par une progression significative dans la mise en place d'une solution complète visant à remplacer l'usage des fichiers Excel au sein de l'équipe hardware. Mon travail s'est articulé autour de plusieurs axes :

- **Finalisation et optimisation de l'API**, afin d'améliorer sa stabilité, sa sécurité et ses performances.
- **Développement d'une première interface web**, intuitive et ergonomique, permettant aux membres de l'équipe de tester et d'exploiter facilement les fonctionnalités de l'API.
- **Mise en place d'outils de validation et de tests**, pour garantir la fiabilité de l'ensemble du système.

Ce rapport s'attachera dans un premier temps à présenter l'évolution de la mission et les améliorations apportées à l'API. Il détaillera ensuite la conception et la mise en œuvre de la première version du site, en soulignant les choix techniques réalisés ainsi que les défis rencontrés. Enfin, il se conclura par un bilan personnel de cette deuxième année d'apprentissage, mettant en avant les compétences acquises et les perspectives pour la suite du projet.

I. Présentation générale de l'entreprise

Fondée en 1981 comme filiale logicielle de Dassault Aviation, **Dassault Systèmes** est rapidement devenue un leader mondial des solutions de conception 3D, de maquettes numériques et de gestion du cycle de vie produit (PLM).

Basée à **Vélizy-Villacoublay (Île-de-France)**, l'entreprise emploie aujourd'hui plus de **20 000 collaborateurs** dans plus de **140 pays**, au service de centaines de milliers de clients issus de multiples secteurs industriels.

Mission : accompagner la transformation digitale des entreprises en proposant des outils innovants pour **concevoir, simuler et optimiser** produits et processus, de l'aéronautique aux biens de consommation.

II. Évolution socio-économique

1. Croissance internationale et diversification sectorielle

- **Années 1980-1990 :**
Démarrage centré sur l'aéronautique, adoption rapide par l'automobile et l'énergie.
 - **Années 2000 :**
Expansion en Asie-Pacifique et Amériques ; acquisitions stratégiques (ex. *SolidWorks* en 1997) pour enrichir l'offre PLM.
 - **Aujourd'hui :**
Présence renforcée dans la santé, la construction, les services publics et les biens de consommation, grâce à la plateforme **3DEXPERIENCE**.
-

2. Impact sur l'emploi et les compétences

- Création de **milliers d'emplois qualifiés** (R&D, ingénierie logicielle, services), moteur de l'essor des métiers du numérique.
- Développement de **partenariats avec universités et écoles d'ingénieurs** pour former la main-d'œuvre aux technologies 3D, simulation et data science.

3. Contributions à la compétitivité industrielle

- **Automatisation et optimisation** des processus de conception et fabrication → réduction des délais et coûts de développement.
 - Déploiement de la **maintenance prédictive** et des **jumeaux numériques**, améliorant la performance opérationnelle et la durabilité.
-

4. Responsabilité sociétale et développement durable

- Engagements pour **réduire l'empreinte carbone**, via la simulation énergétique des bâtiments et l'optimisation des réseaux.
- Initiatives pour l'**inclusion numérique** : programmes de formation et licences éducatives accessibles dans les pays en développement.

III – Présentation d'équipe

1. Service d'accueil

Mon service d'accueil chez **Dassault Systèmes** est spécialisé dans la **certification des logiciels selon différentes configurations matérielles**.

1.1. Missions principales

- Tester les logiciels sur diverses combinaisons de :
 - processeurs, cartes graphiques (GPU), systèmes d'exploitation,
 - versions de logiciels, environnements de développement (IDE) et langages (ex. C++).
- Réaliser chaque année un **processus rigoureux de certification** des machines utilisées par nos clients et nos collaborateurs internes.

1.2 Objectifs du processus

- Garantir la **compatibilité et la performance** des logiciels sur les configurations testées.
- Produire un document annuel appelé "**define**", qui compile les résultats et évalue l'aptitude des machines à exécuter nos solutions.
- Fournir aux clients et partenaires des **directives claires sur les configurations matérielles recommandées** pour une performance optimale.

1.3 Importance et accessibilité

- La certification contribue à **assurer la fiabilité et la qualité** des produits Dassault Systèmes.
- Elle renforce la **confiance des clients** dans nos solutions.

- Les documents de certification sont disponibles sur le site officiel, via la section **Support Hardware and Software**.
-

2. Organigramme de l'équipe

2.1 Structure générale

L'équipe est divisée en deux branches principales :

- **Hardware :**
 - Réception et gestion du matériel (machines complètes ou composants, ex. cartes graphiques).
 - Stockage dans un vaste pool de plusieurs centaines de machines en fonctionnement permanent, permettant des tests continus.
- **Software :**
 - Réalisation des tests logiciels appelés **ODT**.
 - Détection et remontée des erreurs (appelées **KAO**).
 - Correction des anomalies en interne ou collaboration avec les développeurs concernés.

2.2 Mon rôle spécifique

Je participe à l'amélioration du **processus de certification côté hardware**.

- Situation actuelle : chaque nouvelle machine est enregistrée dans un fichier Excel, une méthode peu efficace pour retrouver rapidement une machine.
- Solution proposée : développement d'un **site web interne** permettant de localiser chaque machine à l'aide d'un **code-barres**.
- Objectifs :

- Simplifier la gestion du parc de machines,
- Accélérer la recherche et la récupération des informations,
- Améliorer globalement l'efficacité et la fiabilité du processus de certification.

IV. Mes missions

1. Contexte et Enjeux

Contexte de la mission

L'équipe hardware de Dassault Systèmes utilise depuis plusieurs années des fichiers Excel pour gérer la certification de ses composants et matériels. Bien que pratique pour des volumes réduits, cette solution est devenue inadaptée :

- Les fichiers étaient **multipliés** et difficiles à maintenir.
- Les **erreurs humaines** (copier-coller, doublons, suppression involontaire) impactaient la fiabilité des données.
- L'absence de **sécurité avancée** exposait les données à des accès non autorisés.
- Les **processus manuels** (envoi de fichiers par email, versions locales) ralentissaient la productivité.

Enjeux pour l'entreprise

- **Sécuriser les données** de certification (critiques pour la conformité qualité et légale).
- **Améliorer la productivité** des équipes en centralisant l'information.
- **Réduire les erreurs** liées à la manipulation manuelle d'Excel.

- **Standardiser les accès** via un système unique d'authentification (CAS, déjà utilisé par Dassault Systèmes).
- **Préparer une architecture évolutive** permettant l'intégration avec d'autres systèmes internes.

2. Objectifs opérationnels

Les objectifs de ma mission ont été formulés en termes d'outils et de technologies concrètes :

- **Base de données PostgreSQL** : migration des données Excel → base relationnelle robuste.
- **API REST en Node.js/Express** : gestion CRUD des certifications (Create, Read, Update, Delete).
- **Middleware de sécurité** : intégration avec CAS + génération de tokens.
- **Validation des données** : regex pour emails et téléphones, contraintes d'unicité dans PostgreSQL.
- **Interface web (frontend)** : simplification des tests API et mise à disposition des données.
- **Journalisation** : logs pour suivre toutes les actions sur l'API.

3. Moyens et Ressources

- **Langages & frameworks :**

Backend : Node.js + Express

- **Node.js** : environnement d'exécution JavaScript côté serveur, idéal pour développer des applications performantes et scalables. Son modèle asynchrone et non bloquant permet de gérer efficacement un grand nombre de requêtes simultanées.
- **Express** : framework minimaliste basé sur Node.js, facilitant la création et la gestion des routes, middlewares et API REST. Il offre une structure claire pour la communication entre le front-end et la base de données.

Frontend : React

- Utilisé pour développer le prototype du site web.
- React permet de créer une interface utilisateur dynamique et réactive grâce à son système de composants réutilisables.
- Son approche "state management" rend l'affichage des informations (par exemple la localisation des machines) rapide et fluide.

Base de données : PostgreSQL

- Base de données relationnelle robuste et open-source.
- Particulièrement adaptée pour gérer des données structurées et complexes (ex. inventaire des machines avec leurs configurations, emplacements et statuts).
- Supporte des transactions fiables et sécurisées, garantissant l'intégrité des données dans le processus de certification.

- **Bibliothèques clés :**

- **axios** : utilisée pour la communication entre le front-end et le système d'authentification CAS (Central Authentication Service). Permet d'envoyer facilement des requêtes HTTP et de gérer les réponses.
- **xml2js** : bibliothèque servant à parser les réponses CAS reçues au format XML, afin de les convertir en objets JavaScript exploitables.
- **crypto** : module natif Node.js utilisé pour la génération et le chiffrement des tokens de sécurité. Garantit l'authentification et la protection des échanges de données.
- **sequelize** : ORM (Object-Relational Mapping) permettant de manipuler PostgreSQL avec du code JavaScript. Facilite les opérations CRUD (Create, Read, Update, Delete) et la gestion des relations entre tables.

○

- **Outils utilisés :**

- **Postman** : outil de test d'API utilisé pour vérifier la validité des routes backend et simuler des scénarios d'utilisation avant intégration front-end.
- **GitLab** : gestion de version du code source, avec mise en place de CI/CD (Intégration Continue / Déploiement Continu) afin d'automatiser les tests et déploiements.
- **Ngrok** : solution pour exposer temporairement le serveur backend en HTTPS, permettant de tester facilement l'intégration entre le front-end et le back-end sans configuration complexe de réseau.

4. Déroulement et Méthodologie

4.1 Mise en place du middleware d'authentification

Toutes les routes sensibles passent par le middleware suivant :

```
const authentication = async (req, res, next) => {
  try {
    const authToken = req.header('Authorization')?.replace('Bearer ', '');
    if (!authToken) {
      return res.status(401).json({ message: "Token manquant." });
    }

    const encryptedToken = encrypt(authToken);
    const user = await Ds_user.findOne({ where: { token: encryptedToken } });

    if (!user) {
      return res.status(401).json({ message: "Token invalide ou expiré." });
    }

    req.user = user; // utilisateur injecté dans la requête
    next();
  } catch (error) {
    res.status(500).json({ message: "Erreur d'authentification." });
  }
};
```

Chaque appel API est ainsi filtré et validé avant d'être traité.

4.2 Intégration du CAS (Central Authentication Service)

Le Central Authentication Service (CAS) est un protocole d'authentification unique (Single Sign-On – SSO) permettant à un utilisateur de se connecter une seule fois et d'accéder à plusieurs applications sécurisées sans devoir ressaisir ses identifiants.

L'objectif principal est de centraliser la gestion des identités et de sécuriser l'accès aux applications internes de l'entreprise.

Le flux d'authentification est le suivant :

1. L'utilisateur tente d'accéder au site → redirection vers le **CAS login**.
2. CAS authentifie l'utilisateur → retourne un **ticket**.
3. Le backend valide le ticket via l'endpoint CAS `serviceValidate`.
4. Si valide → un **token chiffré** est généré et stocké en base.
5. Toutes les requêtes suivantes doivent inclure `Authorization: Bearer <token>`.

Exemple de validation du ticket CAS :

```
const validateCasToken = async (ticket, serviceUrl) => {  
  const casValidateUrl = `https://.../serviceValidate?service=${serviceUrl}&ticket=${ticket}`;  
  const response = await axios.get(casValidateUrl);  
  const userInfo = await parseCasResponse(response.data);  
  return userInfo; // contient trigramme + infos CAS  
};
```

4.3 Sécurité des données

- **Chiffrement des tokens**
- **HTTPS obligatoire** pour toutes les communications.
- **Contrôles d'accès** : seuls certains trigrammes (utilisateurs autorisés) peuvent interagir avec l'API.
- **Journalisation** : chaque requête est loguée (qui, quoi, quand).

4.4 Développement du Frontend et gestion des événements

En parallèle du développement de l'API, j'ai conçu une application **frontend en React** permettant d'interagir avec les données stockées dans la base PostgreSQL via l'API sécurisée.

L'objectif principal était de fournir à l'équipe hardware une interface intuitive pour consulter, filtrer et gérer les ressources (workstations, processeurs, GPU, salles, logs, etc.), tout en respectant les mécanismes d'authentification CAS déjà implémentés côté backend.

Authentification côté frontend

Dès le chargement de l'application, le frontend vérifie la présence d'un **token d'authentification** dans l'URL ou dans le **localStorage**.

- Si aucun token n'est présent → l'utilisateur est redirigé vers le CAS Login.
- Si un token valide est présent → il est stocké et utilisé pour tous les appels API protégés.

Extrait du code :

Ici, le **frontend dépend directement de l'API backend** pour valider le token auprès du CAS.

```
useEffect(() => {
  const checkAuth = async () => {
    const frontUrl = window.location.origin;

    // Récupération éventuelle du token en paramètre d'URL
    const urlParams = new URLSearchParams(window.location.search);
    const tokenInUrl = urlParams.get('token');
    const userInUrl = urlParams.get('username') ?? '';

    if (tokenInUrl) {
      localStorage.setItem('authToken', tokenInUrl);
      localStorage.setItem('username', userInUrl);
      localStorage.setItem('tokenTimestamp', Date.now().toString());
      window.history.replaceState({}, document.title, window.location.pathname);
    }

    const token = localStorage.getItem('authToken');
    if (!token) {
      // Pas connecté → redirection vers CAS
      window.location.href =
        `${API_URL}/api/login?redirectUrl=${encodeURIComponent(frontUrl)}`;
      return;
    }

    try {
      const { data } = await statsService.getActions();
      setActions(data.data || []);
      setUsername(localStorage.getItem('username') || '');
      setIsAuthorized(true);
    } catch (err) {
      console.error('Vérification du token échouée', err);
      localStorage.clear();
      window.location.href =
        `${API_URL}/api/login?redirectUrl=${encodeURIComponent(frontUrl)}`;
    } finally {
      setIsLoading(false);
    }
  };

  checkAuth();
}, []);
```

4.5 Gestion des événements et des données

L'application met en place différents **useEffect** pour :

1. Charger les **logs d'actions** (events) depuis l'API.
2. Charger dynamiquement les données (workstations, GPU, processeurs, salles, etc.) en fonction de l'URL visitée.
3. Mettre à jour les filtres, recherches et tris côté frontend sans recharger la page.

Exemple de chargement dynamique des entités :

```
// Chargement des données selon l'entité consultée
const loadEntityData = async (entityType: string) => {
  try {
    let response;
    switch (entityType) {
      case 'workstations':
        response = await statsService.getWorkstations();
        break;
      case 'gpus':
        response = await statsService.getGpus();
        break;
      case 'processors':
        response = await statsService.getProcessors();
        break;
      case 'rooms':
        response = await statsService.getRooms();
        break;
      case 'owners':
        response = await statsService.getOwners();
        break;
    }
    if (response && response.data) {
      setEntityData(response.data.data);
    }
  } catch (error) {
    console.error('Error loading data:', error);
  }
};

// Déclencheur automatique à chaque changement de route
useEffect(() => {
  const path = window.location.pathname;
  const entityType = path.substring(1);
  if (entityType && ['workstations', 'gpus', 'processors', 'rooms', 'owners'].includes(entityType))
    loadEntityData(entityType);
}, [window.location.pathname]);
```

Intégration avec l'API et gestion des logs

Toutes les actions utilisateur (consultation, ajout, modification, suppression) sont envoyées vers l'API. L'API, de son côté, enregistre un **log horodaté** en base, associé au trigramme de l'utilisateur.

Exemple : lors de la consultation des logs, le frontend envoie une requête protégée

V. Bilan professionnel

Cette deuxième année d'apprentissage chez Dassault Systèmes a marqué une étape clé dans ma progression, car elle m'a permis de **consolider les bases posées l'an dernier** tout en élargissant mes responsabilités techniques et organisationnelles.

Facilités observées

- Ma bonne maîtrise acquise en première année sur **Node.js, Express et PostgreSQL** m'a permis d'aborder sereinement les nouveaux développements.
- L'expérience de l'autonomie, déjà travaillée l'an dernier, a facilité ma montée en compétence sur des sujets plus complexes comme la sécurité et l'intégration CAS.

Difficultés rencontrées et moyens mis en œuvre

- La principale difficulté a été la **mise en place d'une authentification sécurisée via CAS**, avec génération de tokens et chiffrement.
- J'ai aussi dû faire face à la **gestion fine des droits d'accès** et à la **journalisation complète des actions utilisateurs**.
- Pour surmonter ces difficultés, j'ai adopté une démarche itérative : documentation, tests unitaires et fonctionnels, échanges avec mon tuteur et vérification régulière auprès des utilisateurs finaux.

Compétences mobilisées et progression

- **Techniques :**
 - Authentification et sécurité (CAS, tokens, crypto, HTTPS obligatoire).
 - Journalisation et traçabilité des actions.

- Développement **frontend React** plus abouti (filtres, recherches, intégration fluide avec l'API).
- **Transverses :**
 - Rigueur dans la gestion de la sécurité et des données sensibles.
 - Amélioration de ma capacité à travailler sur des projets intégrés, en lien direct avec les besoins des utilisateurs.
- Cette année m'a donné une **crédibilité technique renforcée**, en particulier sur les sujets de sécurité et d'industrialisation logicielle.

Complémentarité école / entreprise

- Les enseignements théoriques suivis en école (programmation avancée, gestion des bases de données, sécurité) m'ont fourni un socle précieux pour aborder les problématiques rencontrées en entreprise.
- L'expérience en entreprise, à l'inverse, m'a permis de donner un sens concret à ces notions, en les appliquant à un **projet stratégique pour Dassault Systèmes**.

Perspectives

- Pour ma troisième année, mon tuteur m'a confié de **nouveaux projets**, dont la **certification logicielle de particuliers**. Ces missions, pouvant durer de **un à trois mois**, me permettront de développer des compétences supplémentaires en **gestion de projet long terme**, en organisation et en planification.
 - Je souhaite aussi continuer à améliorer les outils déjà développés et élargir leur intégration dans l'écosystème Dassault Systèmes.
-

VI. Conclusion

Cette deuxième année d'apprentissage a confirmé et enrichi le travail initié l'an dernier. Alors que ma première année était centrée sur la **conception d'une base, d'une API et d'un prototype frontend**, cette année m'a permis d'aller plus loin en **sécurisant et industrialisant** la solution : intégration du **CAS**, chiffrement des tokens, gestion des droits, mise en place de la **journalisation** et amélioration significative de l'interface utilisateur.

Ces avancées techniques m'ont sensibilisé à des enjeux essentiels pour une entreprise de la taille de Dassault Systèmes : la **sécurité des données**, la **traçabilité des actions** et l'**expérience utilisateur**. Elles m'ont aussi donné une plus grande autonomie et une confiance renforcée dans mes capacités à mener des projets à fort impact.

En conclusion, cette deuxième année a représenté un tournant : je suis passé de la **phase de prototypage** à la **phase de fiabilisation et d'intégration**, ce qui reflète bien ma progression en tant qu'apprenti ingénieur. Je suis désormais enthousiaste à l'idée d'aborder ma troisième année avec de **nouveaux projets ambitieux**, comme la certification logicielle de particuliers, qui me permettront de continuer à évoluer et de confirmer ma place au sein de l'entreprise.